
```
def generate_tree(possible_moves, depth, maxdepth):
    tree = []
    for move in possible_moves:
        move_profile = [move]
        if depth < maxdepth:
            possible_moves2 = possible_moves.copy()
            possible_moves2.remove(move)
            move_profile.append(generate_tree(possible_moves2, depth +
1, maxdepth))
        tree.append(move_profile)
    return(tree)
```

[Listing 10-4:](#) *A function that creates a game tree of a specified depth*

This function, `generate_tree()`, starts out by defining an empty list called `tree`. Then, it iterates over every possible move. For each move, it creates a `move_profile`. At first, the `move_profile` consists only of the move itself. But for branches that are not yet at the lowest depth of the tree, we need to add those moves' children. We add children recursively: we call the `generate_tree()` function again, but now we have removed one move from the `possible_moves` list. Finally, we append the `move_profile` list to the tree.

We can call this function simply, with a couple of lines:

```
allpossible = [[(4,4),(4,3)],[(4,1),(5,1)]]
thetree = generate_tree(allpossible,0,1)
print(thetree)
```

When we run this, we see the following tree:

```
[[[(4, 4), (4, 3)], [[[(4, 1), (5, 1)]]], [(4, 1), (5, 1)], [[[(4, 4), (4,
3)]]]]]
```

Next, we'll make two additions to make our tree more useful: the first records the game score along with the moves, and the second appends a blank list to keep a place for children ([Listing 10-5](#)).

```
def generate_tree(possible_moves, depth, maxdepth, game_so_far):
    tree = []
    for move in possible_moves:
        move_profile = [move]
        game2 = game_so_far.copy()
        game2.append(move)
```
